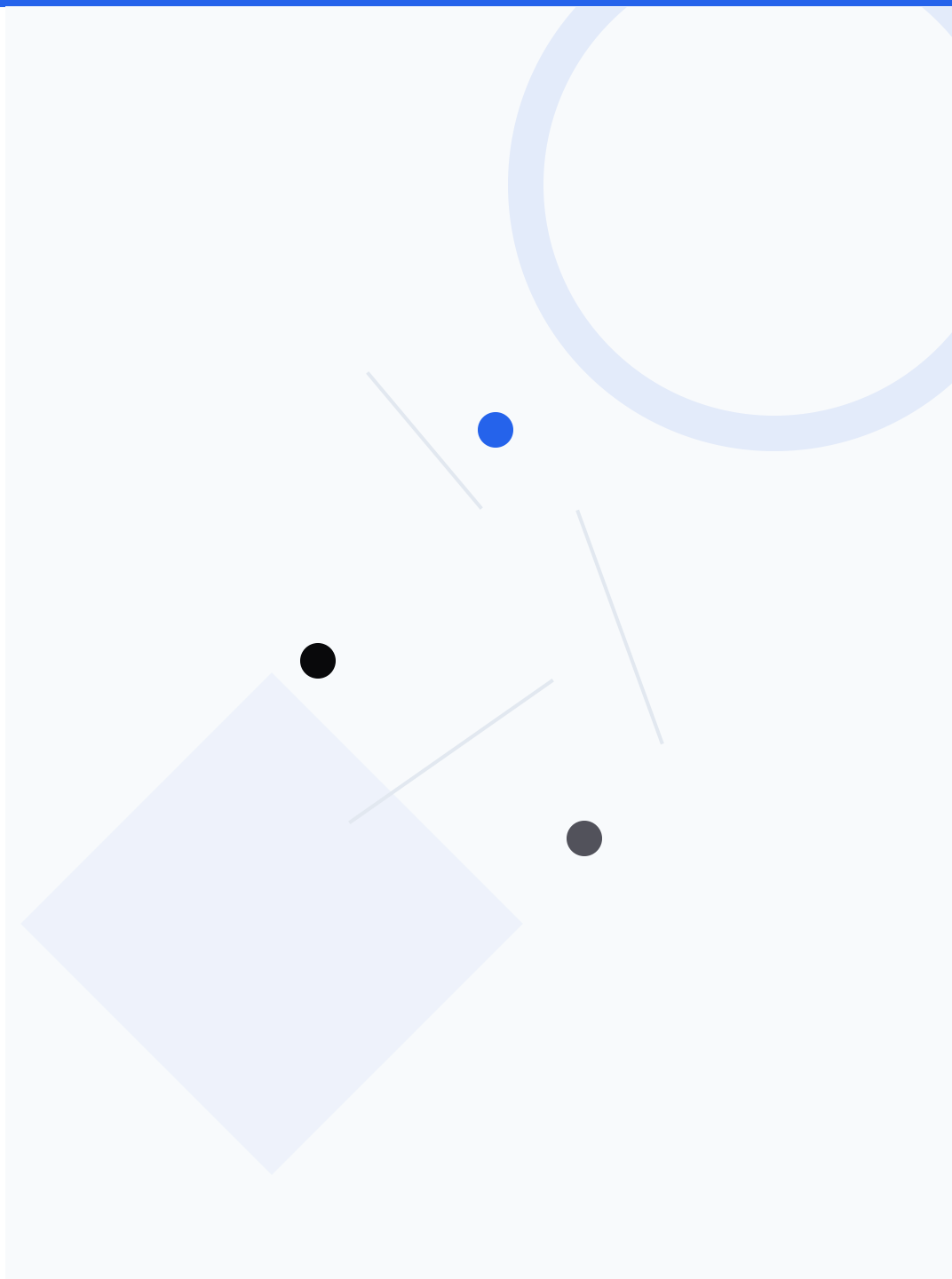


# Hybrid Digital Twin for SDN Networks

## Real-time Topology Synchronization and Active Traffic Emulation

Goal: Build a fully automated system that generates a real-time Digital Twin of an SDN network — replicating physical topology, mirroring active traffic loads, and visualizing the entire system through an interactive web dashboard.



# High-Level Architecture — Five Integrated Layers

The system is composed of five tightly integrated layers working in concert.

| Layer              | Component                     | Role                                |
|--------------------|-------------------------------|-------------------------------------|
| Physical Emulation | Mininet + Ryu SDN Controller  | Emulates the base network           |
| Control Plane API  | Ryu Northbound REST API       | Exposes topology & traffic data     |
| Twin Engine        | Python <code>twin.py</code>   | Fetches data and builds the replica |
| Orchestration      | Shell scripts + Tmux + Docker | Launches all components cleanly     |
| Visualization      | Flask + Socket.IO Dashboard   | Pushes real-time updates to browser |

# The Physical Network & Control Plane

## Baseline Topology

Mininet emulates a linear topology with 3 switches (s1-s2-s3) and 3 hosts (h1, h2, h3).

## Forcing Discovery

`autoStaticArp=True` is intentionally disabled so hosts send real ARP broadcast packets, ensuring the controller discovers the topology organically.

## The Brain — Ryu Controller

Acts as an L2 learning switch, discovering topology via Ryu events and building MAC-to-port mappings dynamically. This live state is then exposed through the Northbound REST API for the Twin Engine to consume.



# The Twin Engine — Solving Namespace Isolation

## The Challenge

Running two Mininet instances on the same Linux host creates kernel namespace conflicts, causing ARP poisoning and routing contamination.

01

### IP Translation

---

Physical IPs ( **10.0.0.x/24** ) are automatically mapped to distinct twin IPs ( **192.168.0.x/24** ), eliminating L3 address collisions.

02

### MAC Remapping

---

Physical MAC prefixes ( **00:00:00:...** ) are translated to a new prefix ( **02:00:00:...** ), ensuring complete L2 isolation.

03

### Port Segregation

---

The twin network uses independent ports — Ryu Controller on 6634 and REST API on 8081 — fully decoupled from the physical plane.

# Dynamic Traffic Emulation — The Key Differentiator

Unlike passive topology mirrors, this twin actively replicates traffic loads in real time:

1

## Polling Statistics

The physical Ryu controller polls **OFPPortStatsRequest** and **OFPFLOWStatsRequest** to calculate real-time Rx/Tx Mbps over time.

2

## Flow Matrix Exposure

Precise bandwidth usage is calculated and exposed via the **/api/flows** REST endpoint.

3

## Active Detection

The Twin engine detects active end-to-end IP flows exceeding 1 Mbps.

4

## Load Injection

It automatically spawns exact **iperf3** client/server replicas between corresponding twin hosts, flawlessly recreating the physical traffic load.

# Continuous Topology Synchronization Loop

The Twin Engine maintains a background polling thread that keeps the replica in perfect sync with the physical network:

- **Polling Interval**

The `/api/topology` endpoint is queried every 10 seconds.

- **Version Checking**

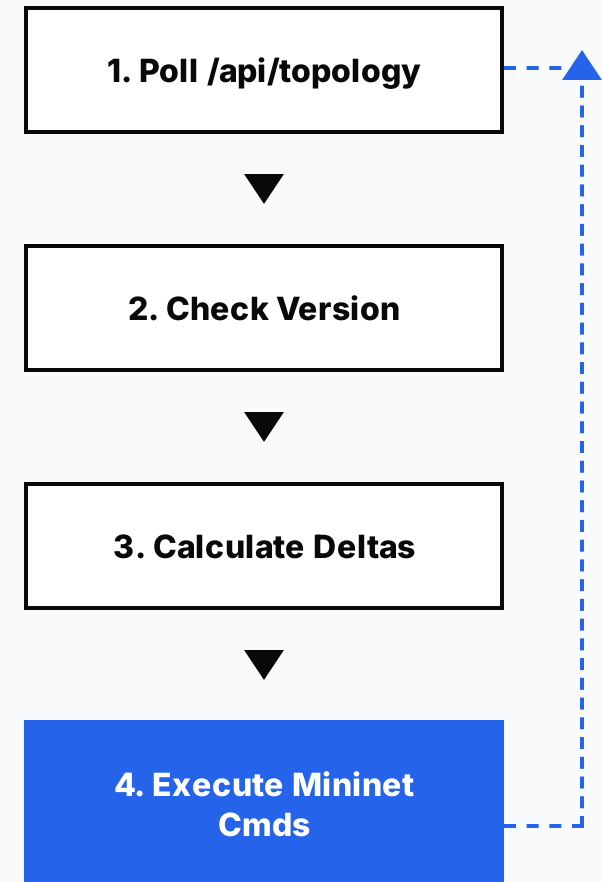
An integer version counter detects state changes efficiently, avoiding heavy reprocessing.

- **Delta Calculation**

Set operations ( $\text{new\_links} - \text{old\_links}$ ) compute the exact topology deltas.

- **Runtime Execution**

Mininet commands are executed on the fly (e.g., interface up/down) to mirror physical events such as a dropped link — with zero manual intervention.



# Real-Time Web Dashboard

The dashboard provides a live, interactive view of both the physical and twin networks simultaneously.

## Backend

Flask application acts as a transparent proxy for Ryu data.

## WebSockets

Socket.IO replaces legacy HTTP polling with persistent, instant state pushes directly to the browser.

## Interactive Graph

The `vis-network` physics engine renders dynamic, draggable network topology graphs.

## Traffic Heatmaps

Edge colors update in real time based on load:

|                                                                                       |               |                      |
|---------------------------------------------------------------------------------------|---------------|----------------------|
|    | <b>Green</b>  | < 1 Mbps (idle)      |
|    | <b>Yellow</b> | 1–5 Mbps (moderate)  |
|  | <b>Red</b>    | > 5 Mbps (congested) |

# Automated Orchestration & Docker Containerization

## THE PROBLEM

---

**Python Compatibility:** The Ryu SDN framework was abandoned in 2017 and is fundamentally incompatible with Python  $\geq 3.12$ , making direct installation on modern systems impossible.

## SOLUTION 1 — DOCKER ISOLATION

---

Ryu is encapsulated inside a custom Docker container running **Python 3.9** with compatible `eventlet` libraries, shielding it from the host environment entirely.

## SOLUTION 2 — TMUX ORCHESTRATION

---

The `start.sh` script automates the launch of **5 separate processes** in split terminal panes — eliminating all manual setup steps and ensuring reproducible, conflict-free deployments every time.



# Current Constraints & Engineering Limitations

Honest assessment of known boundaries:

| Limitation              | Root Cause                                                     | Current Behavior                               |
|-------------------------|----------------------------------------------------------------|------------------------------------------------|
| No runtime host removal | Mininet lacks a clean <code>removeHost()</code> API            | Host teardown not supported at runtime         |
| No host removal events  | Ryu topology module does not reliably emit host removal events | Host state not auto-synced on removal          |
| Dynamic switch changes  | Adding/removing switches requires network restart              | Link state (up/down) is perfectly synchronized |

*These constraints are well-understood and represent clear directions for future engineering work.*

# Conclusion & Live Demonstration

## WHAT WE BUILT

A Hybrid Digital Twin that goes beyond passive topology mapping — it achieves full **namespace isolation**, **active L3 traffic replication**, and **real-time visualization** in a single automated system.

## LIVE DEMO SEQUENCE

- 1** Run `pingall` → populates the dashboard with discovered topology and hosts.
- 2** Trigger `iperf h1 h2` → UI heatmap transitions from Green to Yellow/Red, confirming traffic emulation.
- 3** Execute `link s1 s2 down` → synchronization loop detects the change and drops the corresponding twin link within 10 seconds.

## Key Takeaway

The system demonstrates that a fully automated, namespace-isolated, traffic-aware Digital Twin of an SDN network is not only feasible but deployable on commodity hardware.